

Leveraging Quantum Computing for Database Index Selection

Immanuel Trummer
Cornell University
Ithaca, USA
itrummer@cornell.edu

Davide Venturelli
USRA
Mountain View, USA
ventutech@gmail.com

ABSTRACT

We present an approach to solve the NP-hard database index selection problem on a D-Wave 2X adiabatic quantum annealer with over 1000 qubits. One of our main contributions is an efficient algorithm that maps instances of the index selection problem onto the qubits of the quantum annealer. Our mapping algorithm introduces several novel techniques that have not been used in the context of quantum annealing so far. Compared to prior approaches, we exploit qubits more efficiently which increases the size of the problem instances that can be represented with the given number of qubits by orders of magnitude. Furthermore, our algorithm decreases mapping time by many orders of magnitude compared to prior methods. The fundamental ideas that we present in this paper are applicable to similar optimization problems (e.g., other database related tuning problems). We analyze our algorithm formally and present experimental results on a real quantum annealer located at NASA Ames Research Center in California.

ACM Reference Format:

Immanuel Trummer and Davide Venturelli. 2024. Leveraging Quantum Computing for Database Index Selection. In *Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Q-Data '24)*, June 09–15, 2024, Santiago, AA, Chile. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3665225.3665445>

1 INTRODUCTION

Data sizes keep growing at exponential rates. For a long time, that growth has been matched by exponential increases in speed and energy efficiency of conventional computers. The evolution of conventional computers is however encountering limitations [18]. Denard's law, predicting exponential growth of energy efficiency, has become invalid. Moore's law, predicting exponential growth of circuit density, is predicted to end soon. In view of those developments, it seems wise to explore the potential of alternative computing paradigms to complement classical computers in the long term. This applies in particular to the database community which continuously seeks to push the limits in terms of the data set sizes that can be processed efficiently.

Quantum computing [2] seems currently like a promising candidate to complement conventional computers in the long term. Quantum computers operate on qubits while conventional computers operate on bits. A qubit can be in two states (one and zero)

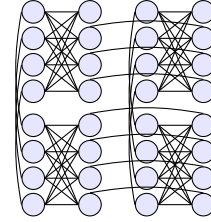


Figure 1: Four neighboring unit cells containing eight qubits each, connected in a Chimera structure.

at the same time that would be considered mutually exclusive according to the laws of classical physics. Such superposition states are however allowed according to the laws of quantum physics. With a highly simplifying intuition, superposition allows quantum computers to exploit a certain type of parallelization that is not possible on classical computers.

Quantum computing has until recently been a purely theoretical research branch of computer science. This has changed in the past few years. While the vision of universal quantum computers with elevated number of qubits may still be years away, first machines implementing at least a limited form of quantum computing have been presented by several companies (e.g., Google [5], IBM [12], and the Canadian company D-Wave Systems [19]). We obtained access to one of those machines, a D-Wave 2X adiabatic quantum annealer with over 1000 qubits. In this paper, we propose and evaluate different methods to use that machine for solving a classical NP-hard optimization problem from the database domain.

The D-Wave quantum annealer used in this paper features a matrix of 1152 qubits (only 1097 of them are functional due to manufacturing errors). Figure 1 shows an extract of that matrix. Qubits are connected pairwise via so called *couplings*. Connections between qubits are sparse and form a distinctive pattern known as the *Chimera graph*: groups of eight qubits form *unit cells*, connected in a bipartite graph structure. Neighboring unit cells are connected via four couplings (assuming fully functional qubits).

The machine can be used to solve NP-hard optimization problems. In order to represent a problem instance, we can choose a weight $w_q \in \mathbb{R}$ for each qubit q and a weight $w_{q_1, q_2} \in \mathbb{R}$ for each coupling connecting qubits q_1 and q_2 . The machine executes a physical process called quantum annealing in order to assign each qubit to a value from the binary domain $\{0, 1\}$ such that the quadratic formula $(\sum_q w_q) + (\sum_{q_1, q_2} w_{q_1, q_2})$ is minimized (we sum over all qubits and couplings). This is a variant of the NP-hard quadratic unconstrained binary optimization (QUBO) problem. Via reduction, we can therefore use the machine to solve any NP-hard problem in principle.



This work is licensed under a Creative Commons Attribution International 4.0 License.

Q-Data '24, June 09–15, 2024, Santiago, AA, Chile
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0553-3/24/06
<https://doi.org/10.1145/3665225.3665445>

In practice, is is challenging to use the machine to solve problems that do not correspond to the native input format. For a given class of problems, we need to find a transformation function that maps instances into the input format required by the quantum annealer. A good transformation function has two qualities: First, it exploits the limited number of qubits very efficiently (e.g., even a quadratic blowup in the problem representation size during the transformation is usually unacceptable). Second, the transformation can be calculated efficiently on a classical computer. Those two goals are at odds, since finding a transformation that uses the minimal number of qubits is again an NP-hard problem in the general case. For that reason, our current focus is on working out efficient transformation functions that are specialized to relevant problem classes in the database domain.

In this paper, we focus on the database index selection problem. Given a workload, a set of index candidates, and a storage space bound, the goal of index selection is to determine an optimal subset of indices to generate. An optimal set of indices does not exceed the storage bound while it optimally benefits the given workload under the latter constraint. We studied the index selection problem for two reasons. First, it is a classical database related optimization problem with a relatively simple structure. Second, it is representative for other tuning related problems in the database domain as it features storage space related constraints, mutual exclusion constraints (as we can select only one clustered index on the same table), and an accumulative benefit metric. Hence, we believe that it is possible to adapt our results for index selection to similar optimization problems in the database area.

We divide the transformation of a given index selection instance into two steps. First, we map the instance to a quadratic formula in binary variables. We construct this formula such that it becomes minimal for an assignment representing an optimal solution to the index selection problem. Second, we map each variable that appears in the quadratic formula to a group of qubits on the annealer matrix to represent it. Qubits representing the same variable have to be connected via couplings (to make sure that all of them are assigned to the same value). As couplings represent quadratic terms, each pair of variables that appears together in a quadratic term needs to be mapped to qubit sets that are connected via at least one coupling. We call the first step *QUBO transformation* and the second step *embedding*.

Our transformation method is based on several key innovations that make solving non-trivial instances of the index selection problem on the quantum annealer practical. Thereby we improve over prior transformation functions that have been proposed for related problems (e.g., the knapsack problem) in several ways. First, we exploit qubits more efficiently which allows us to represent significantly larger problem instances for a given number of qubits. Prior transformation methods for the quantum annealer have mainly focused on reducing the number of binary variables used in the quadratic formula. However, doing so often increases the number of qubits required to represent a single variable. Perhaps counter-intuitively, we find that introducing *additional* auxiliary variables reduces the asymptotic qubit growth from quadratic in the number of indices to linear. Thereby, we are able to represent problem instances with up to 432 index candidates (instead of maximally

Pre-Processing

```
1: // Generate embedding templates for annealer A.
2: global embeddingLib ← GENERATETEMPLATES(A)
```

Run Time

```
3: // Solve index selection problem P using annealer A.
4: function SELECTINDICES(A, P)
5:   // Translate P into quadratic formula to minimize
6:   qubo ← ToQUBO(P)
7:   // Assign QUBO variables to groups of qubits
8:   embedding ← EMBED(A, P, embeddingLib)
9:   // Solve via quantum annealing
10:  indices ← SOLVEBYQA(A, P, qubo, embedding)
11:  // Return solution
12:  return indices
13: end function
```

Algorithm 1: High-level algorithm for solving database index selection on a quantum annealer.

47 with prior methods) on a fully functional matrix of the current annealer model.

Second, we calculate the transformation function itself more efficiently. Efficiency is particularly problematic for the embedding step as finding admissible assignments from variables to qubit groups is an NP-hard problem. Heuristic embedding algorithms exist but work poorly in our case as we demonstrate in our experiments. We make run time embedding efficient via extensive pre-processing. Algorithm 1 shows our high-level approach. During pre-processing before run time, we generate a library of embedding templates. Each embedding template covers a subset of index selection problem instances. Furthermore, templates are specific to the given, concrete annealer machine. In particular, they adapt to the machine-specific pattern of broken and functional qubits (Figure 1 shows an idealized case without broken qubits). At run time, given a concrete problem instance, we select the most suitable template from our library. Generating templates may take a long time but instantiating templates for a given instance is very efficient (i.e., complexity is bounded by a low-order polynomial).

In summary, the original scientific contributions of this paper are the following. First, we describe several reductions from index selection to quadratic unconstrained binary optimization. Second, we propose a method for generating embedding templates for index selection and for efficiently instantiating templates at run time. Third, we formally analyze transformation functions in terms of computational efficiency and representation size (i.e., asymptotic number of required qubits). Finally, we experimentally compare different transformations and embedding algorithms. Also, we report on experiments performed on a real quantum annealer with test cases based on the TPC-H benchmark.

2 FORMAL MODEL

An index is an auxiliary data structure that facilitates data access. Generating indices on a data set can speed up query processing. On the other hand, indices consume storage space so only a limited number of indices can be generated. This leads to the index selection problem: given a database, an example workload of queries on that

data, and a storage threshold, the goal is to identify an optimal set of indices to generate. An optimal set of indices minimizes average query response time for the given workload while the accumulated storage space consumption of all generated indices is below the given storage threshold.

Solving an instance of the index selection problem usually involves two phases [10]. In the first phase, a set of interesting index candidates is identified via heuristics based on the example workload (i.e., index candidates become interesting since they refer to columns that are used in the example workload). In the second phase, the optimal subset of the interesting candidates is identified via cost-based optimization. Choosing an algorithm for the first phase and choosing one for the second are up to a certain extent orthogonal decisions (i.e., we can combine many previously proposed methods for identifying index candidates with many methods for selecting a candidate subset). Hence, prior work often focuses on improving the state of the art only in one of the two phases. In particular, as selecting an optimal index subset is an NP-hard problem [10], the second phase has received significant attention in the database community over the past years. In this paper, we focus on the selection of index candidates as well.

Assuming that a suitable set of index candidates has been previously generated, we formalize the index selection problem as follows. We denote the set of index candidates by I , the storage space consumed when generating index $i \in I$ by s_i , and the threshold for the storage space by b . We denote the utility of generating index $i \in I$ by u_i and assume that the utility of all generated indices accumulates additively. The latter assumption is of course a simplification as different index candidates may overlap, reducing accumulated utility. Nevertheless, this assumption is often made to simplify the problem [10, 27]. Sophisticated methods have been proposed to calculate utility in a manner that partially takes into account overlap between indices [10]. Such methods for estimating utility can be combined with the approach for selecting indices proposed in this paper.

Indexes can be classified into clustered (also called primary) and non-clustered (also called secondary) indexes. A clustered index, as opposed to a non-clustered index, orders data in the same way as the database table it refers to. Whether an index is clustered or not is therefore not a property of the index itself but rather of the database table that the index facilitates access to. We have to choose one tuple order per table so, out of all indices referring to the same table, only one of them can be clustered. We denote the set of database tables that indices refer to by T and we denote for each table $t \in T$ by $C_t \subseteq I$ the set of clustered indices that refer to t .

An optimal solution to an index selection problem instance is therefore a subset $I^* \subseteq I$ such that no two clustered indices are selected for the same table ($\forall t \in T : |C_t \cap I^*| \leq 1$), the storage constraints are respected ($\sum_{i \in I^*} s_i \leq b$), and the accumulated utility ($\sum_{i \in I^*} u_i$) is maximized under the aforementioned constraints.

3 QUBO TRANSFORMATION

The quantum annealer minimizes quadratic formulas that depend on binary variables (i.e., it solves QUBO problems). In order to solve an instance of the index selection problem on the quantum annealer, we first need to transform it into a quadratic formula to

minimize. We show in the following how to transform an instance of the index selection problem into a quadratic formula E of the following form¹:

$$E = E_U + w_C \cdot (E_M + E_S) \quad (1)$$

Term E_U represents the accumulated utility of the selected indices. It takes smaller values the more useful the selected indices are (as the annealer strives to minimize the given formula). Terms E_M and E_S represent constraints restricting the set of selected indices. Term E_M represents mutual exclusion constraints between indices. It takes high values if exclusion constraints are violated (by selecting two indices at the same time with an exclusion constraint between them). Term E_S represents storage constraints. It takes high values if the accumulated storage space consumption of the selected indices exceeds the available storage space. Factor w_C is chosen sufficiently high to make sure that the goal of satisfying all problem constraints is prioritized over the goal of maximizing utility.

All terms depend on the following binary variables: for each index $i \in I$, we introduce a variable g_i representing the choice of selecting that index for generation (i.e., setting $g_i = 1$ means that index i is generated while $g_i = 0$ otherwise). We introduce more variables later. Next, we show how to define the terms in (1). We define term E_U , representing utility of selected indices, as follows:

$$E_U = - \sum_{i \in I} (u_i \cdot g_i)$$

As justified in Section 2, we assume that utility of all selected indices is accumulated additively. The annealer strives to minimize a given function. Hence, we multiply utility values by -1 such that the minimal value corresponds to maximal utility. In the following, we discuss how to incorporate storage and mutual exclusion constraints (between clustered indices on the same table).

3.1 Storage Space Constraints

In the following, we present two alternative formulas for term E_S (which represents storage space constraints). Those formulas are equivalent in the sense that adding them makes sure that the optimal QUBO solution respects storage constraints. The formulas differ however by the number of qubits that are required to represent them on the annealer. The number of qubits is very limited on current annealer models. Finding problem representations that use qubits efficiently is therefore crucial in order to represent non-trivial instances. While we present alternative storage constraint representations in the following, we give a first intuition for why their qubit requirements differ.

Our first representation of storage constraints is relatively similar to the one proposed for the knapsack problem [17]. We denote by s_i the storage space consumed by index i and by $b \in \mathbb{N}^+$ the storage bound. Further, we divide the storage bound b into a set of storage space fractions $f_k \in \mathbb{N}^+$ for $k \in K$ such that two conditions are satisfied. First, the sum over all storage fractions yields the storage bound ($\sum_{k \in K} f_k = b$). Second, each admissible storage space consumption between zero and b (inclusive) can be represented

¹It is common to use symbol E for that formula as its values are internally translated into energy levels on the annealer.

by a sum of the form $\sum_{k \in K} (c_k \cdot f_k)$ with $c_k \in \{0, 1\}$. Choosing ascending powers of two for the storage fractions, except for the last one if the storage bound is no power of two, satisfies both conditions (e.g., choose $f_1 = 1$, $f_2 = 2$, $f_3 = 4$, $f_4 = 3$ for storage bound $b = 10$). The following formula reaches its minimum iff the accumulated storage space consumption of all generated indices does not exceed the bound:

$$E_S^{(1)} = \left(\sum_{i \in I} (s_i \cdot g_i) - \sum_{k \in K} (f_k \cdot c_k) \right)^2$$

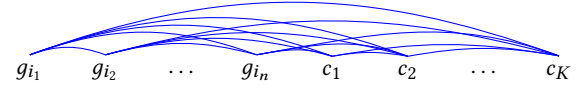
This formula depends on the binary variables g_i , representing index generation, and on the binary variables c_k , representing the consumption of the k -th storage fraction. It is non-negative and reaches its minimum of zero iff the storage space required by the generated indices, $\sum_{i \in I} (s_i \cdot g_i)$, is matched by a combination of storage bound fractions, $\sum_{k \in K} (c_k \cdot f_k)$. As we can only represent storage space consumption that falls in the admissible domain, the minimum of the term is reached only if the storage space required by the generated indices is at or below the bound.

Next, we present a second version of the storage constraint that reduces the number of qubits required to represent problem instances of a given size very significantly. It may at first seem counter-intuitive why the second version improves over the first, since the second version actually uses more variables. However, the number of required qubits is not proportional to the number of variables as the number of qubits required per variable may vary. More precisely, each qubit in the qubit matrix (see Figure 1) is only connected to other qubits via at most six couplings. Only couplings can represent quadratic products between two binary variables. Hence, the number of products between binary variables provides an asymptotic lower bound for the required number of qubits. Decomposing the first representation of the storage space constraints leads to a number of quadratic terms that grows quadratically in the number of index candidates and storage fractions.

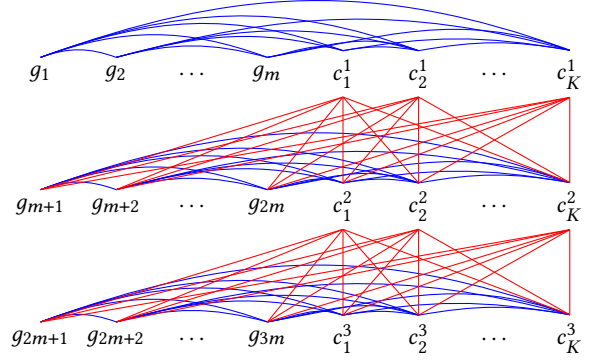
For the second version, we divide indices into disjoint partitions that we denote by $I_1, \dots, I_L$. We change the set of consumption variables to the set c_k^j . The subscript refers, as before, to the storage fraction. The superscript is new and designates an index partition. Setting c_k^j to one means that indices in partitions I_1 to I_j consumed the k -th storage fraction f_k . In order to simplify the following formula, we use the convention $c_k^0 = 0$ for each $k \in K$. The second version of the storage constraint is semantically equivalent to the first:

$$E_S^{(2)} = \sum_{j=1}^J \left(\sum_{i \in I_j} (s_i \cdot g_i) + \sum_{k \in K} (f_k \cdot (c_k^{j-1} - c_k^j)) \right)^2$$

The outer sum above consists of terms that refer to different index partitions. Each term has a minimal value of zero that can only be reached if for the associated partition (partition I_j for the j -th sum term) the accumulated storage space consumed by indices in that partition (determined by variables g_i for indices $i \in I_j$) and the storage space of indices selected in previous partitions (represented by variables $c_k^1, \dots, c_k^{j-1}$) is below the storage bound. At the same time, reaching the minimal value in the j -th term



(a) Representing $E_S^{(1)}$ requires dense connections between all variables.



(b) Representing $E_S^{(2)}$ requires dense connections between all variables of same index partition only.

Figure 2: Comparison of required connections between problem variables for representing different versions of the storage space constraint.

requires to set variables c_k^j for each $k \in K$ to values that correspond to their semantics.

The benefit of the second representation lies in the fact that it requires less dense connections (i.e., two variables are connected if they appear together in a quadratic term) between variables. The connection structures are compared in Figure 2. For the second representation, as we only need to connect the variables of consecutive index partitions, the number of required couplings (and therefore the required number of qubits) grows only linearly in the number of indices if we choose constant partition sizes. We further illustrate the different representation for storage constraints by means of an example.

Example 3.1. Let $b = 3$ be the storage bound, six indices i_1 to i_6 are available. We assume that all indices consume one unit of storage space (e.g., megabytes). We compare different representations in terms of the auxiliary variables they introduce. The first representation divides the storage bound into two fractions, $f_1 = 1$ and $f_2 = 2$, and introduces two associated variables, c_1 and c_2 . The term $1 \cdot c_1 + 2 \cdot c_2$ represents storage consumption. If three indices are selected then an optimal solution assigns c_2 and c_1 both to one. The second representation divides indices into partitions. Assume that we use three partitions where the first partition contains i_1 and i_2 , the second one i_3 and i_4 , while the last one contains i_5 and i_6 . The third representation introduces six auxiliary variables in total, c_k^j where $k \in \{1, 2\}$ designates the fraction and $j \in \{1, 2, 3\}$ the partition. Note that $c_k^0 = 0$ by convention, we do not introduce variables c_k^j for $j = 0$. Assume that one index is selected in the first partition, two indices in the second, and no index in the last one.

Then an optimal QUBO solution sets c_1^1 to one and c_2^1 to zero. This represents a storage consumption of one after taking into account selections in the first partition. An optimal solution sets c_1^2 and c_2^2 both to one to represent a storage consumption of three after taking into account selections in the first two partitions. As no indices are selected in the third partition, we also set $c_1^3 = c_2^3 = 1$.

3.2 Mutual Exclusion Constraints

A clustered index is a special type of index ordering records in the same order as they are stored on disc. There can be at most one clustered index per table. We need to take that constraint into account when selecting indices.

Term E_M represents mutual exclusion constraints between clustered indices that refer to the same table. We present two different versions of E_M in the following. Denoting for each table $t \in T$ by $C_t \subseteq I$ the set of clustered indices that refer to it, we can express E_M as follows:

$$E_M^{(1)} = \frac{1}{2} \cdot \sum_{t \in T} \left(\sum_{i_1, i_2 \in C_t} (g_{i_1}^t \cdot g_{i_2}^t) \right)$$

Term E_M must take high values if exclusion constraints are violated. Using the formula above, term E_M evaluates to a value of at least one if two mutually exclusive indices $i_1, i_2 \in C_t$ are selected. It evaluates to zero otherwise.

As discussed before, the number of qubits required to represent a single variable is proportional to the number of products in which that variable appears. In our first version of the exclusion constraint, the number of products grows quadratically in the number of clustered indices per table. Next, we propose an alternative formulation of the exclusion constraint where that is not the case.

We introduce additional notation to represent the following variant more easily. Assume that we number the clustered indices for each table t in arbitrary order from one to $|C_t|$. Let g_n^t be the binary variable representing the generation of clustered index number n within C_t . Note that this is just a different label referring to a subset of the variables that we introduced before (the ones representing clustered as opposed to nonclustered indices). Furthermore, we introduce binary variables of the form $g_{1..n}^t$ with the following semantics: iff any of the clustered indices in C_t with numbers between (inclusive) one and n was selected then the variable should be set to one.

The second formulation of E_M is based on the following idea. Given the aforementioned ordering between clustered indices on the same table, we introduce for each clustered index a term taking high values if that index and at least one index with lower number are selected at the same time. More precisely, we use variable $g_{1..n-1}^t$ representing the selection of at least one index with a number lower than n . The term $g_{1..n-1}^t \cdot g_n^t$ behaves as described. As we introduce such a term for each clustered index for a given table t , the sum of terms reaches minimal value only if at most one clustered index is selected. Next, we describe how to constrain variables of the form $g_{1..n}^t$ to their appropriate values.

Obviously, it is $g_{1..n}^t = g_1^t \vee \dots \vee g_n^t$. Alternatively, we can use the following recursive definition: it is $g_{1..1}^t = g_1^t$ and we have $g_{1..n}^t = g_{1..n-1}^t \vee g_n^t$ for all $n \in \{1, \dots, |C_t|\}$. Our second variant of the term E_M is based on the latter (recursive) formulation. As we

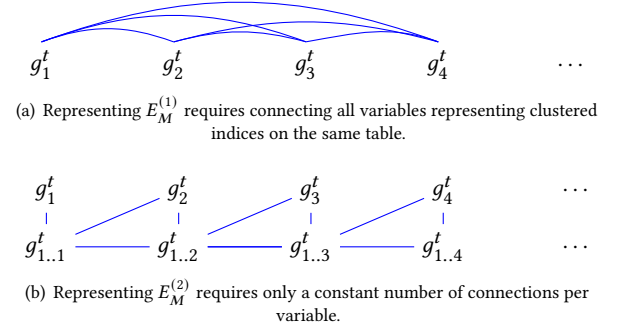


Figure 3: Comparison of connection structure for different versions of the mutual exclusion constraint.

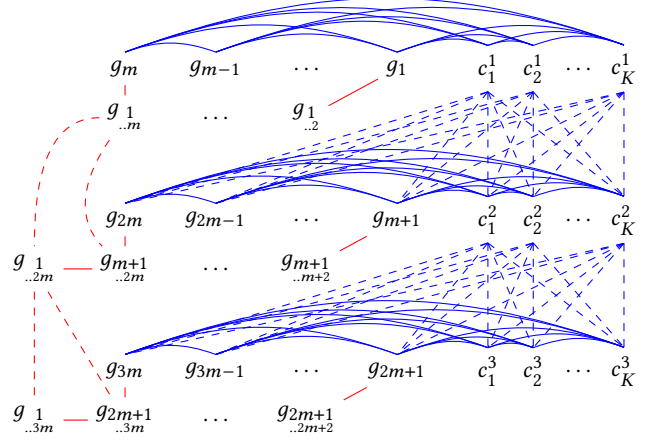


Figure 4: Combined QUBO Connection structure.

cannot directly represent equalities such as $g_{1..n}^t = g_{1..n-1}^t \vee g_n^t$, we must define a term that takes high values if the equality is not satisfied. In the following, we denote by $E[x \vee y = z]$ a term that takes high values if the equality $x \vee y = z$ is not satisfied (and value zero otherwise). The following formula has that property:

$$E[x \vee y = z] = x + y + z + xy - 2xz - 2yz$$

We can now provide the complete formula for the second version of the exclusion constraint term:

$$E_M^{(2)} = \sum_{t \in T} \left(\sum_{n=2}^{|C_t|} (E[g_{1..n-1}^t \vee g_n^t = g_{1..n}^t] + g_n^t \cdot g_{1..n-1}^t) \right)$$

The formula above contains the actual exclusion terms ($g_n^t \cdot g_{1..n-1}^t$) as well as the terms enforcing the right values for variables $g_{1..n}^t$. Unlike the first version, the number of product terms grows not quadratically in the number of clustered indices anymore. Figure 3 illustrates the differences in the connection structure.

3.3 Putting It Together

In this subsection, we integrate all of the aforementioned constraints into one formula. Figure 4 shows the resulting connection structure. Connections that relate to storage space constraints are drawn in blue. Connections that relate to mutual exclusion constraints are drawn in red. Connections between indices in the same partition are drawn using solid lines while connections between different index partitions are represented using dashed lines. No connections are required to represent the objective function. In the next section, Section 4, we present a method for mapping variables to qubit groups such that all required connections are satisfied. The method considers only a restricted family of connection graphs consisting of a sequence of variable clusters. Each variable is associated with one specific cluster and belongs to one out of two types: intra cluster variables are connected to all other variables in the same cluster. Inter cluster variables are connected to all variables in their cluster but also to all variables in the next cluster. Looking at Figure 4, it becomes clear that this family of embedding graphs subsumes all possibly required connection structures: variables that are associated with indices in the same partition belong to the same cluster. Variables at the end of dashed connections are represented as inter cluster variables while all other variables are represented as intra cluster variables. The connection structure shown in Figure 4 applies to an index selection problem with three index partitions that are all connected via mutual exclusion constraints. In many cases, we can simplify the connection structure and reduce the number of variables. In particular, connecting variables of the form g_x across different index partitions is only necessary if mutual exclusion constraints span different index partitions. Even more, if all indices connected by a mutual exclusion constraint belong to the same index partition then we can represent that constraint using the first (i.e., $E_M^{(1)}$) instead of the second representation (i.e., $E_M^{(2)}$) from Section 3.2. The drawback of the first representation was the fact that connections between all related variables are required. But connections between all variables in the same index partition are required anyway to represent storage constraints. Therefore, using the first representation in the same index partition does not require additional connections and reduces at the same time the number of auxiliary variables to create. When integrating all terms into the QUBO formula then we must choose factor w_C to express the relative importance of constraint related terms. We choose w_C as follows: $w_C = \alpha_C \cdot \max_{i \in I} u_i$.

Factor w_C must be sufficiently high to guarantee that formula E is minimized for a selection of indices that satisfies all problem constraints. Selecting a new index i decreases E by a value that is at most proportional to its utility u_i . Choosing w_C higher than the maximal utility over all indices satisfies the main requirement on w_C . Choosing α_C slightly higher than one is therefore in theory sufficient. In practice, choosing α_C too close to one can lead to problems due to limited precision of the annealer when representing weight values. We empirically optimize α_C in Section 5.

All previously described equations refer to logical variables. Logical variables are however represented by groups of qubits. To force all qubits representing the same variable to the same value, we further add terms of the form $\alpha_E \cdot (q_1 + q_2 - 2 \cdot q_1 \cdot q_2)$ between qubits q_1 and q_2 representing the same variable. This term is minimized

once q_1 and q_2 adopt the same value. The choice of α_E must prioritize equality constraints over all other objectives. We empirically optimize α_E in Section 5 as well.

4 PHYSICAL EMBEDDING

An embedding assigns each logical variable (in the QUBO formula from the last section) to a group of qubits representing it. We cannot choose arbitrary qubits for a given variable. First, qubits representing the same variable must be connected via couplings (i.e., a path of couplings must connect any two qubits in the group). Those connections are required to ensure that all qubits representing the same variable take finally the same value. Second, if two variables appear in a quadratic term in the QUBO formula then (at least) one qubit representing the first variable must be connected to one representing the second.

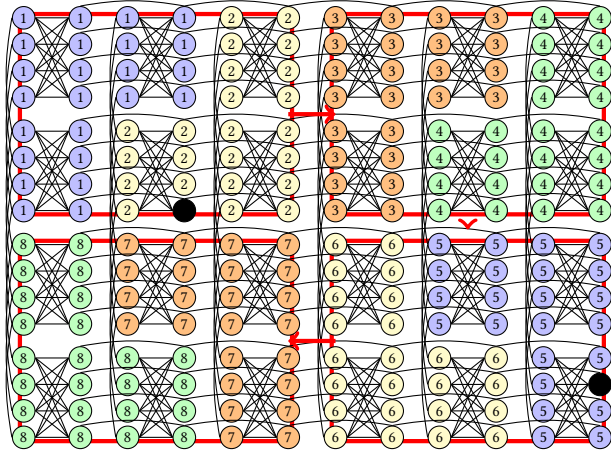
Under those constraints, finding an embedding that uses the minimal number of qubits is an NP-hard problem [11]. We must avoid solving that problem at run time. On the other hand, minimizing the number of required qubits is crucial as the available number of qubits is on current annealers very limited. We solve that dilemma by a two-stage embedding algorithm: the first stage solves multiple NP-hard problems and is executed before run time as a pre-processing step. The second stage is executed at run time. It uses the results of the pre-processing step.

The goal of embedding is to assign variables to qubits groups. We cannot do so at pre-processing time as the precise set of required variables is not yet known. Instead, we identify at pre-processing time groups of qubits that can represent certain types of variables. For instance, we identify groups of qubits that can represent variables modeling the selection of an index candidate. We also identify groups of qubits that can represent variables modeling the consumption of a storage fraction. By doing so, we exploit the relatively regular connection structure between variables (see Figure 4) that is known a-priori within certain limits. At run time, we exploit the mapping between qubit groups and variable types to quickly assign concrete variables. We discuss pre-processing in Sections 4.1 and 4.2 and run time embedding in Section 4.3.

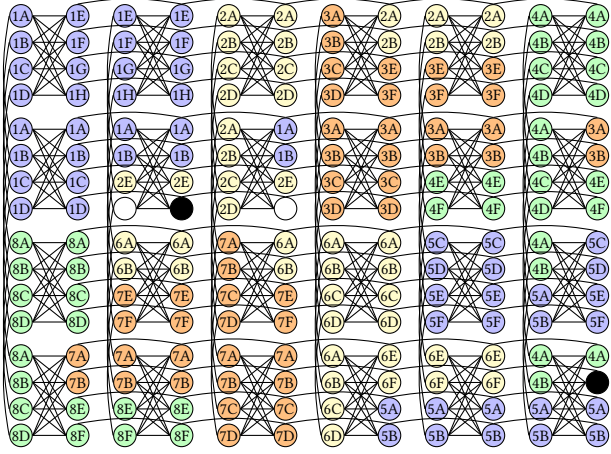
4.1 Pre-Processing: Overview

The goal of pre-processing is to assign qubit groups to variable types. Pre-processing results in general in multiple templates (i.e., mappings between qubits and variable types) from which we select one for each incoming problem at run time. Pre-processing solves multiple NP-hard optimization problems and may take a long time (hours or even days on a commodity computer). We consider that overhead however as unproblematic as pre-processing is executed only once per annealing machine. Compared to other overheads during the setup of a quantum annealer (that might require months of calibration to be made reliable), pre-processing time is moderate. Note that it is insufficient to execute pre-processing for specific annealer models: we need to execute this step for each specific annealer machine as the set of working qubits and couplings can differ between different annealers of the same model.

Our pre-processing algorithm consists of two steps that are illustrated in Figures 5(a) and 5(b). In the first step (Figure 5(a)),



(a) We partition working qubits in each box (box in red) into two triangles of a given width (here: two unit cells) such that consecutive triangles share many connections.



(b) We group qubits in each triangle to represent maximal number of fully connected variables (A to H), two variables (A and B) connect consecutive clusters.

Figure 5: We generate embedding templates in two steps: after partitioning qubits into triangle-shaped blocks (Figure 5(a)), we group qubits to represent a chain of connected variable clusters (Figure 5(b)).

we partition all working qubits into a chain of connected, triangle-shaped formations (each red box in Figure 5(a) contains two of those triangles and qubits belonging to the same triangle have the same color and number). The choice of triangle shapes and the positioning of the triangles is justified by the variable connection structure that we ultimately want to represent. As shown in Figure 4, we require clusters of fully connected variables that are connected in a chain of clusters. Triangles are ideal shapes to represent fully-connected clusters if no broken qubits are present (this is not the case in general but we assume that triangles are still a good choice as long as the number of broken qubits is not excessive). Packing two triangles as shown into each box and placing boxes in a zig-zag pattern ensures the maximal number of couplings between the

qubits of consecutive triangles (at least if no broken couplings are present).

The second part of the pre-processing algorithm, illustrated in Figure 5(b), is based on the qubit partitioning generated in the first. We treat triangles in ascending order: for each triangle, we carve out qubit groups representing a fully connected variable cluster with a specified number of connections to the previous cluster. We call variables connecting consecutive clusters (i.e., a variable that is connected to all other variables in two consecutive clusters) inter-cluster variables and the other variables intra-cluster variables. Matching the resulting connection structure to the required connections depicted in Figure 4, we find that variables of type intra-cluster variable can represent index selections (variables g in Figure 4) while variables of type inter-cluster variable can represent any logical variable. We exploit that classification when mapping qubit groups to concrete variables at run time.

Assigning variables in a specific cluster to groups of qubits in a specific triangle would be trivial if all qubits and couplings would be intact. Alas, manufacturing errors are common and assigning qubits within each triangle becomes a hard problem. A-priori, we cannot know how many fully connected variables can be embedded in each triangle. Our goal is to carve out a cluster of maximal size from each triangle while realizing the required number of connections to the previous cluster. We do so by translating the local embedding problems (i.e., assigning the qubits of single triangles) into binary integer linear programming problems. We solve the resulting problems using standard solvers such as CPLEX. This may take a long time as the resulting problems are NP-hard. However, as justified before, the overhead of pre-processing is moderate compared to other annealer setup costs.

The pre-processing algorithm that was described so far is parameterized by two parameters: the size of the triangles (measured as the width and height in unit cells) and the number of required connections between consecutive clusters. For a given annealer machine, we execute pre-processing for all possible parameter values. Each parameter value combination results in one embedding template that is stored in the library. At run time, the most suitable template from the library is selected for each incoming problem.

4.2 Pre-Processing: Details

We describe the pre-processing step in more detail. Algorithm 2 partitions qubits into a sequence of triangle-shaped formations (illustrated in Figure 5(a)). Algorithm 3 is based on the result from Algorithm 2 and forms connected clusters of maximal size from the qubits in each triangle.

We discuss the partitioning algorithm first. Function `PARTITION-QUBITS` in Algorithm 2 takes a description A of an annealer machine as input and the triangle width w . The result is a list of qubit sets where each set forms an isosceles triangle with width and height (measured as the number of involved unit cells) w on the qubit matrix. Furthermore, the qubits in consecutive triangles tend to share many connections in the qubit matrix.

Function `PARTITION-QUBITS` starts by dividing the two-dimensional qubit matrix into rows and columns such that each cell (shown as red boxes in Figure 5(a)) can accommodate two triangles. We iterate

```

1: // Returns qubits of annealer  $A$  in a triangle of unit cells
2: // of width  $w$  with apex at corner  $apexCorner$  of a
3: // box with upper-left corner coordinates  $(bx, by)$ .
4: function TRIANGLE( $A, w, bx, by, apexCorner$ )
5:   // Calculate lower-right box coordinates
6:    $bx2 \leftarrow bx + w$ 
7:    $by2 \leftarrow by + w - 1$ 
8:   // Get coordinates of cells in triangle
9:   if  $apexCorner = UL$  then
10:     $C \leftarrow \{(x, y) | x \geq bx, y \geq by, x - bx + y - by < w\}$ 
11:   else if  $apexCorner = LL$  then
12:     $C \leftarrow \{(x, y) | x \geq bx, y \leq by2, x - bx + by - y < w\}$ 
13:   else if  $apexCorner = LR$  then
14:     $C \leftarrow \{(x, y) | x \leq bx2, y \geq by2, bx - x + by - y < w\}$ 
15:   else if  $apexCorner = UR$  then
16:     $C \leftarrow \{(x, y) | x \leq bx2, y \leq by, bx - x + y - by < w\}$ 
17:   end if
18:   // Return all qubits in those unit cells
19:   return  $\cup_{(x,y) \in C} A.qubitsInCell(x, y)$ 
20: end function

21: // Partition qubits on annealer  $A$  into adjacent
22: // triangle formations with cell width  $w$ .
23: function PARTITIONQUBITS( $A, w$ )
24:   // Initialize triangle counter
25:    $tCtr \leftarrow 0$ 
26:   // Calculate box dimensions
27:    $boxWidth \leftarrow w + 1$ 
28:    $boxHeight \leftarrow w$ 
29:   // Divide qubit matrix into rows and columns
30:    $nrRows \leftarrow \lfloor A.maxCellX / boxWidth \rfloor$ 
31:    $nrColumns \leftarrow \lfloor A.maxCellY / boxHeight \rfloor$ 
32:   // Iterate over qubit rows
33:   for  $rowCtr \leftarrow 1 \dots nrRows$  do
34:     // Y coordinate of box corner
35:      $by \leftarrow (rowCtr - 1) \cdot boxHeight$ 
36:     // Iterate over qubit columns
37:     for  $columnCtr \leftarrow 1 \dots nrColumns$  do
38:       // Specific partitioning for even/uneven rows
39:       if  $rowCtr \bmod 2 = 1$  then
40:         // X coordinate of box corner
41:          $bx \leftarrow (columnCtr - 1) \cdot boxWidth$ 
42:         // Partition qubits within box
43:          $trias[tCtr] \leftarrow \text{TRIANGLE}(A, w, bx, by, UL)$ 
44:          $tCtr \leftarrow tCtr + 1$ 
45:          $trias[tCtr] \leftarrow \text{TRIANGLE}(A, w, bx, by, LR)$ 
46:          $tCtr \leftarrow tCtr + 1$ 
47:       else
48:         // X coordinate of box corner
49:          $maxX \leftarrow nrColumns \cdot boxWidth$ 
50:          $bx \leftarrow maxX - columnCtr \cdot boxWidth$ 
51:         // Partition qubits within box
52:          $trias[tCtr] \leftarrow \text{TRIANGLE}(A, w, bx, by, UR)$ 
53:          $tCtr \leftarrow tCtr + 1$ 
54:          $trias[tCtr] \leftarrow \text{TRIANGLE}(A, w, bx, by, LL)$ 
55:          $tCtr \leftarrow tCtr + 1$ 
56:       end if
57:     end for
58:   end for
59:   // Return chain of triangle-shaped qubit formations
60:   return  $trias$ 
61: end function

```

Algorithm 2: Partitioning qubits into a chain of adjacent triangle formations.

over the cells in a zig-zag pattern: we iterate from left to right for un-even row numbers and from right to left for even row numbers. For each cell, we partition all working qubits within that cell's bounding box into two triangles. Measuring width and height of those triangles by the number of unit cells, those triangles are isosceles and their apexes are at opposite sides of the bounding box. For un-even row numbers, the first triangle has its apex at the north-west

```

1: // Embed maximal number of fully connected variables in
2: // qubits of  $t$ -th triangle in  $trias$  while assuring  $co$  connections
3: // on annealer  $A$  to qubits of  $t + 1$ -th triangle and to  $t - 1$ -th
4: // set of intra-cluster variables in  $intras$ .
5: function MAXCLUSTER( $A, trias, co, t, intras$ )
6:   // Embed largest possible cluster
7:   Find maximal  $s$  and  $Q[1], \dots, Q[2 \cdot co + s]$ 
   Such That
   //  $Q[*]$  are disjunct subsets of qubits in  $t$ -th triangle
    $\forall x : Q[x] \subseteq trias[t], \forall x \neq y : Q[x] \cap Q[y] = \emptyset$ 
   // Realize connections to previous cluster if any
    $t = 0 \vee \forall 1 \leq x \leq co : A.connected(intras[t - 1, x], Q[x])$ 
   // Realize connections to next triangle if any
    $t = \text{LENGTH}(trias) - 1 \vee$ 
    $\forall co < x \leq 2 \cdot co : A.connected(Q[x], trias[t + 1])$ 
   // Variables form fully connected cluster
    $\forall x \neq y : A.connected(Q[x], Q[y])$ 
8:   // (Connects to previous cluster, to next, unconnected)
9:   return  $\langle Q[1..co], Q[co + 1..2 \cdot co], Q[2 \cdot co + 1..2 \cdot co + s] \rangle$ 
10: end function

11: // Given triangle partitioning  $trias$  for qubits of annealer  $A$ ,
12: // group qubits of each triangle to represent variables in fully
13: // connected clusters with  $co$  variables connecting
14: // consecutive clusters.
15: function FORMCLUSTERS( $A, trias, co$ )
16:   // Initialize qubits of inter-cluster variables
17:    $inters[*] \leftarrow \emptyset$ 
18:   // Initialize qubits of intra-cluster variables
19:    $intras[*] \leftarrow \emptyset$ 
20:   // Iterate over all triangles
21:   for  $t \leftarrow 0, \dots, \text{LENGTH}(trias) - 1$  do
22:     // Form largest possible cluster from triangle qubits
23:      $\langle P, N, U \rangle \leftarrow \text{MAXCLUSTER}(A, trias, co, t, intras)$ 
24:     // Add qubits connecting to previous triangle
25:      $intras[t, *] \leftarrow intras[t, *] \cup P[*]$ 
26:     // Add qubits connecting to next triangle
27:      $intras[t, *] \leftarrow N[*]$ 
28:     // Add unconnected qubits
29:      $inters[t, *] \leftarrow U[*]$ 
30:   end for
31:   // Return qubits for inter- and intra-cluster variables
32:   return  $\langle inters, intras \rangle$ 
33: end function

```

Algorithm 3: Embed largest possible variable cluster in each triangle while assuring required number of connections between consecutive clusters.

corner and the second triangle has its apex at the south-east corner of the bounding box. For even row numbers, the south-west and north-east corner are used for the apexes.

Algorithm 3 assigns the qubits in each triangle (produced in the previous step) to represent variables in a chain of fully-connected clusters. Function FORMCLUSTERS in Algorithm 3 takes as input the annealer description A , the list of triangles, $trias$, and the minimal number of required connections co between consecutive clusters. The output is a tuple containing two arrays. The first array, $inters$, stores sets of qubits that can represent variables of inter-cluster type (i.e., a variable that is connected to all variables in two consecutive clusters). The second array, $intras$, stores sets of qubits that can represent variables of intra-cluster type (i.e., variables that are connected to all other variables in the same variable cluster). Both arrays are two-dimensional where the first index represents the cluster number and the second index represents the variable number.

Function FORMCLUSTERS iterates over the triangles produced by the previous algorithm. It tries to form fully connected clusters

of maximal size (measured by the number of variables) from the qubits in each triangle. It iterates over all triangles and uses sub-function `MAXCLUSTER` to assign the qubits in a single triangle to variables. Each call to function `MAXCLUSTER` returns a three-tuple of arrays storing sets of qubits from the current triangle. The first returned array, P , stores qubit sets representing variables of inter-cluster type that connect to the previous triangle. The second returned array, N , stores qubit sets representing variables of inter-cluster type that connect to the next triangle. The third array, U , stores qubit sets representing variables of intra-cluster type. They are unconnected (i.e., they need not be connected to either the previous or the next triangle). We add qubit sets in P and N to the result variable $inter$ storing qubit sets of inter-cluster variables. Qubit sets in U are added to the corresponding fields of result array $intras$, representing intra-cluster variables. We use the asterisk to represent each admissible index value (e.g., $intras[t, *] \leftarrow intras[t, *] \cup P[i]$ adds the qubits in $P[i]$ to the qubits in $intras[c, i]$ for each admissible index i).

The assignment from triangle qubits to variables, realized by function `MAXCLUSTER`, must satisfy various constraints. We need to assign each variable to a connected group of qubits. As our goal is to form a fully-connected cluster, qubit groups representing different variables must share at least one connection. Further, as the qubits representing intra-cluster variables are distributed over two consecutive triangles, variables representing intra-cluster variables in the current triangle must be connected to their counterparts in the previous triangle. Function `MAXCLUSTER` translates the problem of assigning triangle qubits to variables, a local version of the embedding problem, into a binary integer linear program (ILP). We solve the resulting ILP using integer programming solvers such as CPLEX.

4.3 Run Time Embedding

The result of pre-processing is a library of embedding templates. Each template assigns groups of qubits to variable types (i.e., inter and intra cluster variables). At run time, we choose the best template for a given problem instance.

Then, we use that template to efficiently assign concrete variables to qubits. Algorithm 4 shows pseudo-code for the run time component of our embedding algorithm. Function `EMBED` is invoked for each incoming index selection problem instance. Its output is a function (i.e., the embedding) mapping each QUBO variable to a qubit group. We treat variables representing storage space consumption first (i.e., variables of the form c_x^y in the formulas in Section 3) by invoking the auxiliary function `EMBEDSPACEVARS`. Then we treat variables representing the generation of index candidates (i.e., variables of the form g_x^y in Section 3). We use the auxiliary procedure `EMBEDINDEXSET` to embed variables associated with subsets of indices. Clustered indices are treated first, ordered by the database table they refer to. Non-clustered indices are treated afterwards.

The order in which indices are embedded is crucial. Procedure `EMBEDINDEXSET` uses up one template cluster after the other when assigning qubit groups to variables. This means that only those index variables that are assigned during the same call of `EMBEDINDEXSET` are guaranteed to be assigned to consecutive template

```

1: // Extend embedding E for selection problem P by variables
2: // representing space consumption using template l.
3: procedure EMBEDSPACEVARS(P, l, E)
4:   // Calculate required number of space vars
5:   nrVars  $\leftarrow \lceil \log(P.b) \rceil$ 
6:   // Iterate over all template clusters
7:   for cc  $\leftarrow 1 \dots \text{LENGTH}(inters)$  do
8:     // Embed each space variable
9:     for k  $\leftarrow 1 \dots nrVars$  do
10:      EMBEDINTER(E,  $c_k^{cc}$ , l, cc)
11:     end for
12:   end for
13: end procedure

14: // Extend embedding E by indices in set S using
15: // clusters starting at cluster number cc in template l.
16: procedure EMBEDINDEXSET(S, l, E, cc, clustered)
17:   // Until entire index set is embedded
18:   while S  $\neq \emptyset$  do
19:     // How many qubit groups in current cluster?
20:     nrGroups  $\leftarrow \text{LENGTH}(l.intras[cc])$ 
21:     // Track index selections in current cluster?
22:     if clustered  $\wedge |S| > nrGroups$  then
23:       // Select indices to embed
24:        $\langle i_1, \dots, i_{nrGroups/2} \rangle \subseteq S$ 
25:       // Embed selected indices
26:       for gc  $\leftarrow 1, \dots, nrGroups/2$  do
27:         EMBEDINTRA(E,  $g_{i_{ctr}}$ , l, cc)
28:       end for
29:       // Embed aggregate variables
30:       for ctr  $\leftarrow 1, \dots, nrGroups/2 - 1$  do
31:         EMBEDINTRA(E,  $g_{i_1 \dots i_{ctr}}$ , l, cc)
32:       end for
33:       // Embed last aggregate variable
34:       EMBEDINTER(E,  $g_{i_1 \dots i_{nrGroups/2}}$ , l, cc)
35:     else
36:       // Select indices to embed
37:        $\langle i_1, \dots, i_{nrGroups} \rangle \subseteq S$ 
38:       // Embed selected indices
39:       for ctr  $\leftarrow 1, \dots, nrGroups$  do
40:         EMBEDINTRA(E,  $g_{i_{ctr}}$ , l, cc)
41:       end for
42:     end if
43:     // Remove embedded indices
44:     S  $\leftarrow S \setminus \{i \in P.I \mid \exists Q : \langle g_i, Q \rangle \in E\}$ 
45:     // Advance template cluster counter
46:     cc  $\leftarrow cc + 1$ 
47:   end while
48: end procedure

49: // Assign QUBO variables for index selection problem P
50: // to qubit groups using embedding templates in L.
51: function EMBED(P, L)
52:   // Initialize embedding
53:   E  $\leftarrow \emptyset$ 
54:   // Initialize template cluster counter
55:   cc  $\leftarrow 0$ 
56:   // Select embedding template
57:   l  $\leftarrow \text{SELECTTEMPLATE}(P, L)$ 
58:   // Embed variables related to space
59:   EMBEDSPACEVARS(P, l, E)
60:   // Iterate over database tables
61:   for t  $\in T$  do
62:     // Embed all clustered indices for current table
63:     EMBEDINDEXSET(P.Ct, l, E, cc, true)
64:   end for
65:   // Embed all non-clustered indices
66:   EMBEDINDEXSET(P.I  $\setminus \cup_t P.Ct$ , l, E, cc, false)
67:   // Return embedding
68:   return E
69: end function

```

Algorithm 4: Efficiently constructing an embedding at run time by exploiting a library of embedding templates.

clusters. Variables representing clustered indexes on the same table must be connected to enforce mutual exclusion constraints. As only consecutive clusters are connected, such variables must be assigned in one call of the auxiliary function. Next, we discuss the pseudo-code of the auxiliary functions in more detail.

Procedure `EMBEDSPACEVARS` embeds variables of the form c_x^y representing storage space consumption. We assume that all parameters are call-by-reference parameters. Hence, embedding E , specified as parameter, can be changed by the invoked procedure. First, the procedure calculates the number of required space consumption variables (which depends on the storage threshold $P.b$). Then, the procedure embeds those variables in each template cluster. Note that, even though we embed capacity variables in each cluster, we might only require a subset of them depending on the number of clusters needed to represent all indices.

We use procedure `EMBEDINTER` to embed a single variable in a specific cluster. The call `EMBEDINTER(E, c_k^{cc}, l, cc)` changes the input embedding E by adding one mapping for variable c_k^{cc} . The newly added mapping assigns that variable to the next unused qubit group of type inter cluster that can be found in cluster number cc in template l . We need to assign capacity variables to inter cluster qubit groups such that the amount of consumed storage space can be passed on to the following clusters. We also use procedure `EMBEDINTRA` to assign a new variable to an unused qubit group of type intra cluster in a specified template cluster.

Procedure `EMBEDINDEXSET` extends the input mapping by assigning all variables associated with a given set S of indices to qubit groups in the template. The set of variables associated with the indices depends on whether or not those indices are clustered. The input flag *clustered* indicates whether all indices to embed are clustered indices on the same table.

A-priori, it is unclear how many template clusters are needed to embed the given set of indices. The while loop iterates until all indices are embedded, using in each iteration the qubit groups in a new cluster (variable cc is the cluster counter). In each iteration, we first count the number of intra cluster qubit groups in the current cluster. Which variables we need to embed depends in part on the relationship between the remaining number of indices to embed and the number of available qubit groups.

If we embed non-clustered indices (i.e., flag *clustered* is set to false) then we do not need to represent any mutual exclusion constraints between them. Therefore, we only need to embed a single variable g_i for each index $i \in S$, representing whether that index is generated or not. The same applies if we need to represent mutual exclusion constraints but can fit all remaining indices into the current cluster. As all variables in the same cluster are mutually connected, we can exploit those connections to represent mutual exclusion constraints as well. We need however additional variables to represent mutual exclusion constraints if we cannot embed all remaining indices in the current cluster. In that case, we must create additional variables capturing whether one index in the current cluster was selected and pass on that information to the next cluster. Those additional variables are of the form $g_{i1...i2}$ and indicate whether one out of a certain range of indices was selected.

If we pass on the selection of indices to the next cluster (i.e., if we embed clustered indices and cannot embed all of them in the

current cluster) then we need only one inter cluster qubit group (summarizing whether or not at least one of the indices in the current cluster was selected) while the others can be of type intra cluster. If we do not have to pass on the selection of indices to the next cluster (i.e., if we embed non-clustered indices or if we can embed all clustered indices in the current cluster) then all index-related variables can be assigned to intra cluster groups.

In general, whenever an intra cluster qubit group is needed, an inter cluster qubit group can be used instead. It is straight-forward to extend procedure `EmbedIntra` to consider that possibility. We did not provide details on how the right embedding template is selected for a given index selection problem instance. As discussed in more detail in Section 5, the number of embedding templates generated by our approach is very moderate for current quantum annealers and grows slowly in the available number of qubits. Therefore, a naive but still practical approach is to try the embedding procedure on each template in the library. A template is rejected if a call to one of the procedures `EMBEDINTRA` or `EMBEDINTER` cannot be completed due to a lack of unused qubit groups. A more sophisticated method of selecting embedding templates relies on an index structure that allows to find suitable templates faster: we can index embedding templates for instance based on the number of intra cluster connections and based on the total number of inter cluster and intra cluster groups. For a given problem instance, we can quickly obtain the required number of intra cluster connections (by analyzing the storage bound) and bounds on the required number of total groups. That analysis, combined with the index on embedding templates, reduces the set of templates to consider.

5 EXPERIMENTAL EVALUATION

The goals of our experiments are threefold. First, we show that our QUBO transformation exploits qubits more efficiently than previous transformations (see Section 5.1). Second, we demonstrate that our template-based embedding algorithm is much more efficient than standard embedding algorithms (see Section 5.2). Third, as a proof of concept, we show that we find good approximate solutions for TPC-H [1] based instances of the index selection problem via quantum annealing (see Section 5.3).

5.1 QUBO Transformation

In Section 3, we proposed different QUBO representations for index selection. We justified intuitively that the second representation of both constraints consumes asymptotically less qubits than the first (which is similar to previously proposed representations). Now we quantify that difference for current annealer models and models that are expected in the coming years.

Figure 6 compares the first constraint representation, close to prior work, against the second representation advocated in this paper. We compare the maximal dimensions of index selection instances that can be represented with a given number of qubits. We assume a fully functional matrix of qubits and a perfect embedding algorithm. The number of qubits and couplings on the matrix is therefore the only restricting factor.

Using fewer bits (i.e., variables c_k) to represent storage space consumption enables us to represent more indices. For our annealer model, we can represent up to 432 indices (47 indices with prior

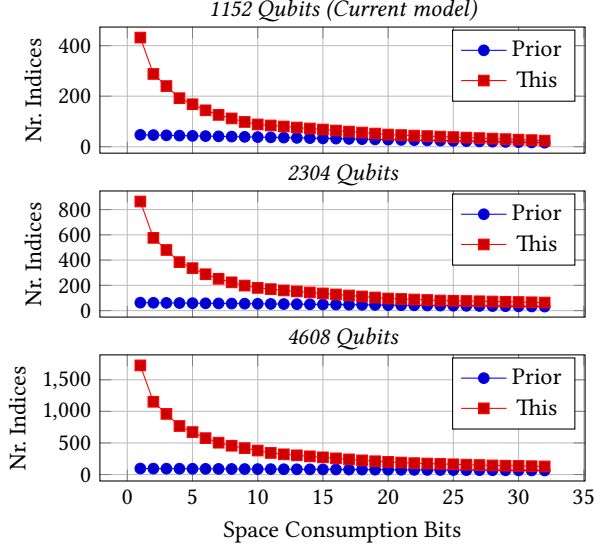


Figure 6: Comparison of maximal problem sizes that can be represented with a given number of qubits.

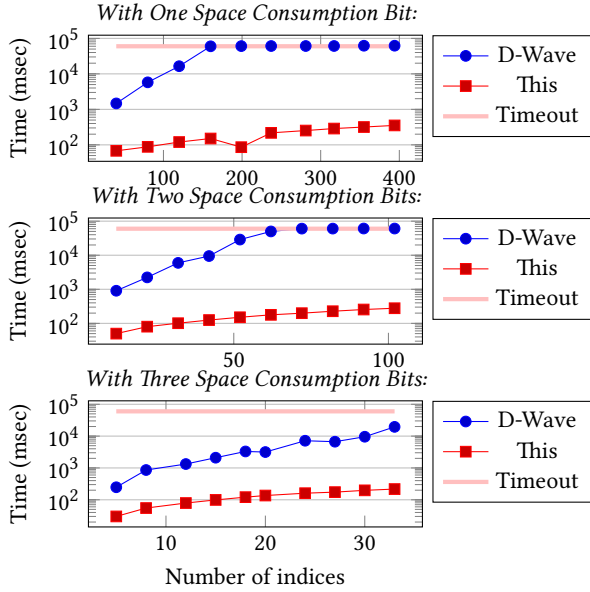


Figure 7: Comparison of embedding times between D-Wave embedder and our algorithm.

methods). Furthermore, the relative distance increases as the number of qubits grows.

5.2 Embedding Algorithm

In Section 4, we proposed a novel embedding algorithm. This algorithm reduces run time embedding overhead by an expensive pre-processing phase (which is however only executed once in the lifetime of an annealer machine).

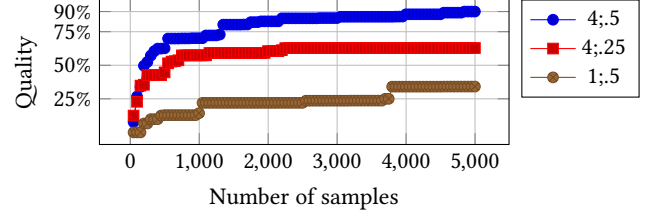


Figure 8: Convergence of solution quality for different term weights in energy formula.

Figure 7 compares run time embedding overhead of our method against the standard embedding algorithm in the software package by D-Wave [8]. The latter algorithm is implemented in C. Its input is a connection structure indicating which variables need to be connected via couplings. Its output is, ideally, a mapping from variables to connected qubit groups that realizes the required connections. The algorithm is however a heuristic and not guaranteed to find solutions to the NP-hard embedding problem.

Each data point in Figure 7 corresponds to the arithmetic average of 10 test cases. The connection structure that forms the input to the D-Wave embedder is the one required for the combined representation of the QUBO problem, described in Section 3.3. Our own algorithm, described in Section 4.3, is implemented in Java 1.7. Using the method described in Section 4.2, we generated seven templates with triangle width between one and three which took around 41.9 hours. We use three of those templates for the experiments in Figure 7 with between one and three intra cluster connections.

We vary the number of space consumption variables and increase the number of indices up to the maximal number that can be represented with the available qubits. We target embedding at the qubit matrix of the annealer at NASA Ames Research Center with 1097 functional qubits. All experiments were executed on a Windows computer with 12 GB of main memory and a 2.5 GHz Intel i7-6500U CPU.

Figure 7 demonstrates the benefit of pre-processing. The standard embedding algorithm quickly reaches the timeout of one minute per test case. Its run time seems to grow exponentially in the number of indices. Our algorithm, despite JVM-related overheads, is by many orders of magnitude faster than the standard embedding algorithm. As we instantiate previously generated templates, run time embedding overheads are always below 360 milliseconds. Furthermore, our code is not optimized for performance and we believe that further gains are possible.

5.3 Quantum Annealing

We solved several test cases on a D-Wave 2X adiabatic quantum annealer with 1097 qubits. Our experiments are based on the TPC-H benchmark and the Postgres database system. Having instantiated the TPC-H database in Postgres, we generated the indices recommended by Fortler and Michel for that specific database [13]. For each index separately, we measured storage space consumption (as the number of pages on disc) and benefit (as the number of seconds

by which its presence reduces execution time for the first ten TPC-H queries). We use those measurements to instantiate an empirical probability distribution over index storage space consumption and benefit. More precisely, assuming that storage space and benefit are correlated, we generate an empirical distribution over benefit values and a second one over the ratio between storage space consumption and benefit. For the following test cases, we generate index candidates according to that distribution. Storage bounds are chosen to accommodate 10% of the candidates (selected randomly) in the given problem instance.

It is a-priori unclear whether it is possible to obtain near-optimal solutions for such test cases on current quantum annealers. First, the quantum annealing process is in general sensitive to outside disturbances. Such disturbances might prevent the annealer from finding optimal solutions and their impact depends on the problem class. Second, our test cases are challenging since they allow a wide range of storage space consumption values. Increasing the precision of the storage space representation consumes however additional qubits (since it requires to introduce additional auxiliary variables) which are rare. Third, our test cases are also challenging since they cover a wide range of index benefit values. Index benefit values translate directly into weights on qubits. Unlike space consumption values, precision is therefore not dependent on the number of qubits. However, the precision with which weights can be represented during quantum annealing is limited. Therefore, we need to verify whether the available precision is sufficient to distinguish beneficial from less beneficial index candidates.

We configured the annealer to execute for each test case 5000 annealing runs of a duration of 20 microseconds. We used a template with three connections between consecutive clusters (three storage consumption bits) and embedded the maximal number of 33 non-clustered index candidates. Figure 8 shows a representative extract from our experimental results. Each data point is the arithmetic average of 10 test cases. We report relative quality (i.e., 100% is a solution with the maximal accumulated benefit, a solution that exceeds the storage space bound has value zero) of the best solution found after a certain number of annealing runs.

There are various ways to tune our transformation algorithm. Based on preliminary results, we decided to use raw index benefit and space consumption values during the transformation (instead of rounding or ceiling values). We scale up constraints by factor four (i.e., $\alpha_C = 4$ in (1)). For consistency constraints, forcing all qubits representing the same variable to the same value, we calculate a conservative scaling factor α_E (similar to Trummer and Koch [26]). Then we divide that overly pessimistic factor by two to reduce the range of weights that need to be represented. The curves in Figure 8 correspond to the latter tuning option, and two alternatives (reducing the constraint weight from four to one and reducing consistency scaling from .5 to .25).

We have only tried a limited number of tuning options. However, the settings described above lead already to valid solutions with an average quality of above 90%. The two alternatives exemplify problems due to suboptimal weight choices: reducing consistency weights leads to solutions in which 18.7% of all variables have inconsistent assignments. Reducing constraint weights leads to solutions that exceed storage bounds in 99.8% of all cases. With

the right setting, we are however able to obtain close to optimal solutions for a class of realistic and challenging test cases.

6 RELATED WORK

Index selection is a classical NP-hard optimization problem from the database domain [3, 9, 10, 20, 27]. We target this problem as it has a relatively simple structure while many of the transformation challenges (e.g., how to represent storage space constraints) are representative for related problems in the same space (e.g., materialized view selection [3]). Prior approaches to index selection include for instance greedy algorithms [27], branch-and-bound approaches [9], and integer programming [20]. Our work is related to prior work showing how to transform index selection into formalisms such as integer programming in order to use software solvers. Our work differs from prior work since we target a novel solver with very specific requirements.

Work on applied problems on the D-Wave machine has proliferated in the last years, with QUBO transformations, embedding approaches, and experimental results presented in the domains of planning and scheduling [23, 28], fault-diagnostics [22], simulation of quantum chemistry [4], machine learning [6], and protein folding [21]. In the domain of databases, prior work targets query optimization [24–26], optimization of transaction schedules [7, 14], as well as unstructured search [15]. The problem of index selection via quantum computing has been considered in another recent work [16], but without presenting experimental results.

We compare the problem of index selection to the one of multiple query optimization [26], the first database-specific optimization problem mapped to a quantum annealer. Index selection is in many ways a more challenging problem to map onto the quantum annealer. First, we have global constraints (i.e., storage bounds) connecting all decision variables (i.e., representing index generations). It is this distinctive problem property, leading to quadratic qubit growth if using naive representations, that motivated us to examine the alternative representations discussed in Section 3. The ideas introduced in Section 3 could also be used to improve the representation of multiple query optimization [26] (e.g., by reducing qubit growth from quadratic to linear when representing exclusion constraints between alternative plans for the same query). Second, long chains of interconnected qubits (resulting from global constraints) make it difficult to avoid broken qubits on the annealer matrix. This motivates us to generate embedding templates that are specialized to the patterns of broken and functional qubits on specific machines (see Section 4). Finally, unlike multiple query optimization, the number of qubits required to represent an index selection instance depends on the weight range in the input problem: the number of qubits required for representing space consumption variables grows in the range of possible space consumption values. It is therefore a-priori unclear whether we find near-optimal solutions to realistic instances with a large range of index sizes.

7 CONCLUSION

We show how to leverage quantum annealers to solve the database index selection problem. Our experimental evaluation on a real quantum annealer shows that our approach finds near-optimal solutions for realistic test cases.

REFERENCES

- [1] [n. d.]. TPC-H Homepage. <https://www.tpc.org/tpch/>
- [2] Scott Aaronson. 2013. *Quantum computing since Democritus*.
- [3] Sanjay Agrawal, Surajit Chaudhuri, and Vivek R. Narasayya. 2000. Automated selection of materialized views and indexes in SQL databases. In *VLDB*. 496–505. <http://portal.acm.org/citation.cfm?id=671701>
- [4] Ryan Babbush, Peter J. Love, and Alán Aspuru-Guzik. 2014. Adiabatic quantum simulation of quantum chemistry. *Scientific Reports* 4 (2014). <https://doi.org/DOI:10.1038/srep06603> arXiv:1311.3967
- [5] R. Barends, A. Shabani, L. Lamata, J. Kelly, A. Mezzacapo, U. Las Heras, R. Babbush, A. G. Fowler, B. Campbell, Yu Chen, Z. Chen, B. Chiaro, A. Dunsworth, E. Jeffrey, E. Lucero, A. Megrant, J. Y. Mutus, M. Neeley, C. Neill, P. J. J. O'Malley, C. Quintana, P. Roushan, D. Sank, A. Vainsencher, J. Wenner, T. C. White, E. Solano, H. Neven, and John M. Martinis. 2016. Digitized adiabatic quantum computing with a superconducting circuit. *Nature* 534, 7606 (2016), 222–226. <https://doi.org/10.1038/nature17658> arXiv:1511.03316
- [6] Marcello Benedetti, John Realpe-Gómez, Rupak Biswas, and Alejandro Perdomo-Ortiz. 2016. Quantum-assisted learning of graphical models with arbitrary pairwise connectivity. *arXiv:1609.02542* (2016), 1–13. arXiv:1609.02542 <http://arxiv.org/abs/1609.02542>
- [7] Tim Bittner and Sven Groppe. 2020. Avoiding blocking by scheduling transactions using quantum annealing. In *ACM International Conference Proceeding Series*. <https://doi.org/10.1145/3410566.3410593>
- [8] J. Cai, B. Macready, and A. Roy. 2014. A practical heuristic for finding graph minors. , 16 pages. arXiv:arXiv:1406.2741v1 <http://arxiv.org/abs/1406.2741>
- [9] Alberto Caprara, Matteo Fischetti, and Dario Maio. 1995. Exact and approximate algorithms for the index selection problem in physical database design. *KDE* 7, 6 (1995), 955–967. <https://doi.org/10.1109/69.476501>
- [10] Surajit Chaudhuri. 2004. Index selection for databases: A hardness study and a principled heuristic solution. *KDE* 16, 11 (2004), 1313–1323. http://ieeexplore.ieee.org/xpls/abs_all.jsp?arnumber=1339260
- [11] Vicky Choi. 2011. Minor-embedding in adiabatic quantum computation: II. Minor-universal graph design. *Quantum Information Processing* 10, 3 (2011), 343–353. <https://doi.org/10.1007/s11128-010-0200-3> arXiv:1001.3116
- [12] Jerry M. Chow, Jay M. Gambetta, A. D. Córcoles, Seth T. Merkel, John A. Smolin, Chad Rigetti, S. Poletto, George A. Keefe, Mary B. Rothwell, J. R. Rozen, Mark B. Ketchen, and M. Steffen. 2012. Universal quantum gate set approaching fault-tolerant thresholds with superconducting qubits. *Physical Review Letters* 109, 6 (2012). <https://doi.org/10.1103/PhysRevLett.109.060501> arXiv:1202.5344
- [13] Paul J. Fortler and Howard Edgar Michel. 2003. *Computer systems performance evaluation and prediction*.
- [14] Sven Groppe and Jinghua Groppe. 2021. Optimizing Transaction Schedules on Universal Quantum Computers via Code Generation for Grover's Search Algorithm. In *ACM International Conference Proceeding Series*. 149–156. <https://doi.org/10.1145/3472163.3472164>
- [15] Lov K Grover. 1997. Quantum computers can search arbitrarily large databases by a single query. *Physical Review Letters* 79, 23 (1997), 4709–4712. <https://doi.org/10.1103/PhysRevLett.79.4709>
- [16] Le Gruenwald, Tobias Winker, Umut Çalıkyılmaz, Jinghua Groppe, and Sven Groppe. 2023. Index Tuning with Machine Learning on Quantum Computers for Large-Scale Database Applications. In *CEUR Workshop Proceedings*, Vol. 3462.
- [17] Andrew Lucas. 2014. Ising formulations of many NP problems. *Frontiers in Physics* 2, 5 (2014). <https://doi.org/10.3389/fphy.2014.00005> arXiv:1302.5843
- [18] Prof Christian Martín. 2014. *Post-Dennard Scaling and the final Years of Moore 's Law consequences for the evolution of multicore-architectures*. Technical Report.
- [19] Catherine C Mcgeoch, Cong Wang, Burnaby Bc, Murray Thom, and Dominic Walliman. 2013. Experimental evaluation of an adiabatic quantum system for combinatorial optimization. In *International Conference on Computing Frontiers*.
- [20] Stratos Papadomanolakis and Anastasia Ailamaki. 2007. An integer linear programming approach to database design. In *ICDEW*. 442–449. <https://doi.org/10.1109/ICDEW.2007.4401027>
- [21] Alejandro Perdomo-Ortiz, Neil Dickson, Marshall Drew-Brook, Geordie Rose, and Alán Aspuru-Guzik. 2012. Finding low-energy conformations of lattice protein models by quantum annealing. *Scientific reports* 2, 1 (2012), 571. <https://doi.org/10.1038/srep00571>
- [22] A. Perdomo-Ortiz, J. Fluegemann, S. Narasimhan, R. Biswas, and V.N. Smelyanskiy. 2015. A quantum annealing approach for fault detection and diagnosis of graph-based systems. *The European Physical Journal Special Topics* 224, 1 (2015), 131–148. <https://doi.org/10.1140/epjst/e2015-02347-y>
- [23] Eleanor G. Rieffel, Davide Venturelli, Bryan O'Gorman, Minh B. Do, Elicia Prystay, and Vadim N. Smelyanskiy. 2014. A case study in programming a quantum annealer for hard operational planning problems. *Quantum Information Processing* 14, 1 (jul 2014), 1–36. arXiv:1407.2887 <http://arxiv.org/abs/1407.2887>
- [24] Manuel Schoenberger, Immanuel Trummer, and Wolfgang Maurer. 2023. Quantum Optimisation of General Join Trees. In *QDSM*. https://www.lfd.r.de/Publications/2023/ScTrMa_QDSM23.pdf
- [25] Manuel Schoenberger, Immanuel Trummer, and Wolfgang Maurer. 2023. Quantum-Inspired Digital Annealing for Join Ordering. *PVLDB* (2023). https://lfd.r.de/Publications/2023/ScTrMa_TR.pdf
- [26] Immanuel Trummer and Christoph Koch. 2016. Multiple Query Optimization on the D-Wave 2X Adiabatic Quantum Computer. *PVLDB* 9, 9 (2016), 648–659. <https://doi.org/10.14778/2947618.2947621>
- [27] Gary Valentin, M. Zuliani, Daniel C. Zilio, Guy M Lohman, and Alan Skelley. 2000. DB2 advisor: An optimizer smart enough to recommend its own indexes. In *ICDE*. 101–110. <https://doi.org/10.1109/ICDE.2000.839397>
- [28] Davide Venturelli, Dominic J. J. Marchand, and Galo Rojo. 2015. Quantum annealing implementation of job-shop scheduling. *arXiv preprint arXiv:1506.08479* (2015). arXiv:1506.08479 <http://arxiv.org/abs/1506.08479>